UNIVERSITÉ DE KARA
Département de Mathématiques
Filière Développement Web et Mobile — 2025/2026

Documents de Conception — Phase 1

Application de Gestion des Réclamations

Ministère de la Justice et de la Législation :

TP Final — Cours Frameworks Python
Année académique : 2025/2026

1. Contexte et Présentation du Projet

Le Ministère de la Justice et de la Législation a amorcé un processus de digitalisation de ses services. Dans ce cadre, une application de gestion des réclamations est développée pour centraliser le suivi des tickets soumis par les citoyens et traités par les techniciens.

1.1 Acteurs du système

Acteur	Rôle
Citoyen / Agent	Soumet des réclamations, suit l'état de ses tickets
Technicien Support	Reçoit les tickets assignés, les traite et met à jour leur statut
Administrateur	Gère les utilisateurs, les rôles, les statistiques et la configuration

1.2 Stack technologique

Couche	Technologie	Rôle
Backend API	Django 4.x + DRF	Modèles, API REST, permissions
Authentification	SimpleJWT	Émission et vérification des tokens JWT
Base de données	PostgreSQL	Stockage persistant
Frontend mobile	Flutter 3.x (Dart)	Interface mobile Android/iOS

HTTP Client	package http (Dart)	Communication Flutter → API
Stockage local	SharedPreferences	Persistance du token JWT côté mobile

2. Diagramme de Cas d'Utilisation

Le diagramme de cas d'utilisation identifie les trois acteurs du système et leurs interactions avec les fonctionnalités de l'application.

2.1 Code PlantUML

Copiez ce code sur plantuml.com pour générer le diagramme :

```
@startuml
left to right direction
skinparam actorStyle awesome
skinparam packageStyle rectangle

actor "Citoyen / Agent" as C
actor "Technicien" as T
actor "Administrateur" as A

rectangle "Système de Gestion des Réclamations" {
  usecase "Créer un compte" as UC1
  usecase "Se connecter" as UC2
  usecase "Soumettre un ticket" as UC3
  usecase "Consulter ses tickets" as UC4
  usecase "Suivre l'état d'un ticket" as UC5
  usecase "Commenter un ticket" as UC6
  usecase "Voir les tickets assignés" as UC7
  usecase "Changer le statut d'un ticket" as UC8
  usecase "Assigner un ticket" as UC9
  usecase "Gérer les utilisateurs" as UC10
  usecase "Voir les statistiques" as UC11
  usecase "Archiver les tickets clos" as UC12
  usecase "Configurer types et priorités" as UC13
}
```

```
C --> UC1
C --> UC2
C --> UC3
C --> UC4
C --> UC5
C --> UC6

T --> UC2
T --> UC6
T --> UC7
T --> UC8
T --> UC9

A --> UC2
A --> UC8
A --> UC9
A --> UC10
A --> UC11
A --> UC12
A --> UC13

@enduml
```

2.2 Description des cas d'utilisation

Cas d'utilisation	Acteur(s)	Description
Créer un compte	Citoyen	S'enregistrer avec email, nom et mot de passe
Se connecter	Tous	Authentification JWT — obtenir access + refresh token

Soumettre un ticket	Citoyen	Créer une réclamation avec titre, type et priorité
Consulter ses tickets	Citoyen	Voir la liste de ses réclamations avec filtres
Suivre l'état	Citoyen	Observer les changements de statut du ticket
Commenter	Tous	Ajouter un commentaire à un ticket
Voir tickets assignés	Technicien	Consulter les tickets attribués et ouverts
Changer le statut	Tech/Admin	Passer le ticket à EN_COURS, RESOLU ou CLOS
Assigner un ticket	Tech/Admin	Attribuer un ticket à un technicien
Gérer les utilisateurs	Admin	CRUD utilisateurs et attribution des rôles
Voir les statistiques	Admin	Tableau de bord : tickets par statut et technicien
Archiver les tickets	Admin	Clôturer automatiquement les tickets résolus
Configurer types/priorités	Admin	Paramétrer les catégories et niveaux de priorité

3. Diagramme de Classes UML

Le diagramme de classes présente les quatre entités principales du système et leurs relations.

3.1 Code PlantUML

```
@startuml
skinparam classAttributeIconSize 0

class CustomUser {
  +id : int
  +email : str
  +username : str
  +first_name : str
  +last_name : str
  +role : Role
  +telephone : str
  --
  +is_technicien() : bool
  +is_admin_role() : bool
}

enum Role {
  CITOYEN
  TECHNICIEN
  ADMIN
}

class Ticket {
  +id : int
  +titre : str
  +description : str
  +type_ticket : TypeTicket
  +statut : Statut
```

```
  +priorite : Priorite
  +date_creation : datetime
  +date_modification : datetime
  +date_resolution : datetime
}

enum TypeTicket {
  INCIDENT
  RECLAMATION
  DEMANDE
}

enum Statut {
  OUVERT
  EN_COURS
  RESOLU
  CLOS
}

enum Priorite {
  BASSE
  NORMALE
  HAUTE
  CRITIQUE
}

class Commentaire {
  +id : int
  +contenu : str
  +date : datetime
}

class HistoriqueStatut {
  +id : int
```

```
  +ancien_statut : str
  +nouveau_statut : str
  +date_changement : datetime
}

CustomUser "1" --> "0..*" Ticket : crée (auteur)
CustomUser "1" --> "0..*" Ticket : est assigné à
CustomUser "1" --> "0..*" Commentaire : rédige
CustomUser "1" --> "0..*" HistoriqueStatut : modifie
Ticket "1" --> "0..*" Commentaire : contient
Ticket "1" --> "0..*" HistoriqueStatut : possède
CustomUser --> Role
Ticket --> TypeTicket
Ticket --> Statut
Ticket --> Priorite

@enduml
```

3.2 Description des entités

Entité	Attributs clés	Description
CustomUser	email, role, telephone	Hérite de AbstractUser — rôle CITOYEN/TECHNICIEN/ADMIN
Ticket	titre, statut, priorite, auteur, assigne_a	Entité centrale — workflow OUVERT→CLOS
Commentaire	contenu, date, ticket, auteur	Messages échangés sur un ticket

HistoriqueStatut	ancien_statut, nouveau_statut, modifie_par	Traçabilité de tous les changements de statut

4. Diagramme de Séquences — Création d'un Ticket

Ce diagramme décrit le flux complet de création d'un ticket depuis l'application Flutter jusqu'à la base de données PostgreSQL, avec gestion de l'authentification JWT et du refresh automatique.

4.1 Code PlantUML

```
@startuml
skinparam sequenceArrowThickness 2
skinparam roundcorner 10

actor Citoyen
participant "Flutter App" as F
participant "API Django (DRF)" as API
participant "SimpleJWT" as JWT
database "PostgreSQL" as DB

== Authentification ==
Citoyen -> F : Saisit email + mot de passe
F -> API : POST /api/auth/login/ {email, password}
API -> JWT : Vérifie credentials
JWT --> API : Génère access + refresh token
API --> F : {access: "...", refresh: "..."}
F -> F : Stocke tokens dans SharedPreferences

== Création du ticket ==
Citoyen -> F : Remplit formulaire ticket
F -> F : Récupère access_token depuis SharedPreferences
F -> API : POST /api/tickets/
note right: Authorization: Bearer <token>
API -> JWT : Vérifie token JWT
JWT --> API : Token valide — utilisateur = Citoyen

alt Token expiré
```

```
  API --> F : 401 Unauthorized
  F -> API : POST /api/auth/refresh/ {refresh: "..."}
  API --> F : {access: "nouveau_token"}
  F -> API : POST /api/tickets/ (retry)
end

API -> API : Valide données (serializer DRF)
API -> DB : INSERT INTO tickets_ticket (auteur,
statut=OUVERT, ...)
DB --> API : Ticket créé (id=X)
API --> F : 201 Created {id, titre, statut: OUVERT}
F --> Citoyen : Affiche confirmation

@enduml
```

4.2 Description du flux

Étape	Acteur → Système	Description
1	Citoyen → Flutter	Saisie des identifiants sur l'écran de connexion
2	Flutter → API	POST /api/auth/login/ avec email et password
3	API → SimpleJWT	Vérification des credentials et génération des tokens
4	API → Flutter	Retour du token access (2h) et refresh (7j)
5	Flutter → Local	Stockage des tokens dans SharedPreferences
6	Citoyen → Flutter	Remplissage du formulaire de création de ticket

7	Flutter → API	POST /api/tickets/ avec header Authorization: Bearer
8	API → SimpleJWT	Vérification du token JWT
8b	Si 401 → Refresh	Flutter appelle /api/auth/refresh/ et relance la requête
9	API → PostgreSQL	INSERT du ticket avec statut OUVERT et auteur = citoyen
10	API → Flutter	201 Created avec les données du ticket créé
11	Flutter → Citoyen	Affichage du message de confirmation